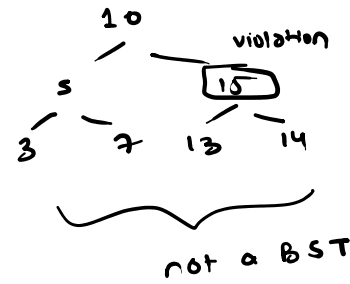
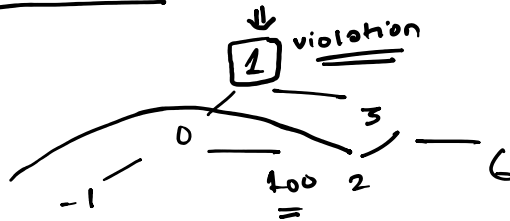
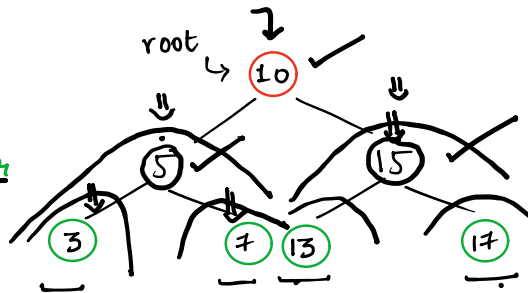


Binary Search Trees (BST)

A binary tree is said to be a BST if the value of every node in the BT is greater than value of all the nodes in its LST and less than value of all the nodes in its RST.

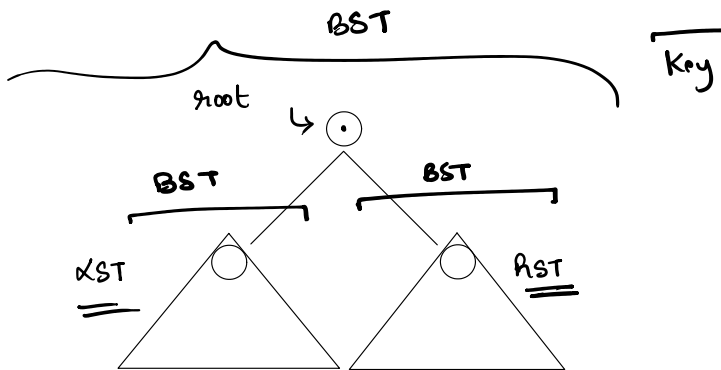
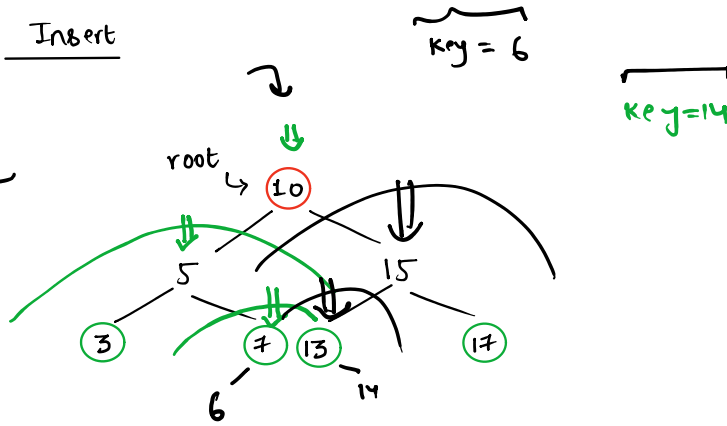


Why do we need a BST?

Search op. in a BST is systematic and efficient.

↳ BST Insert

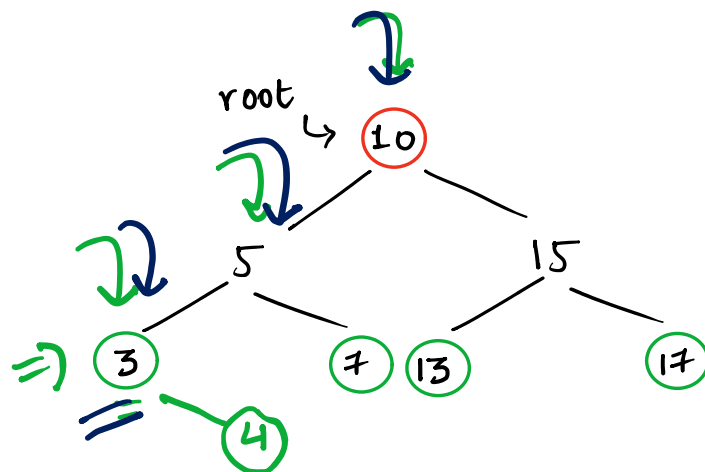
time $\propto O(\text{hgt})$



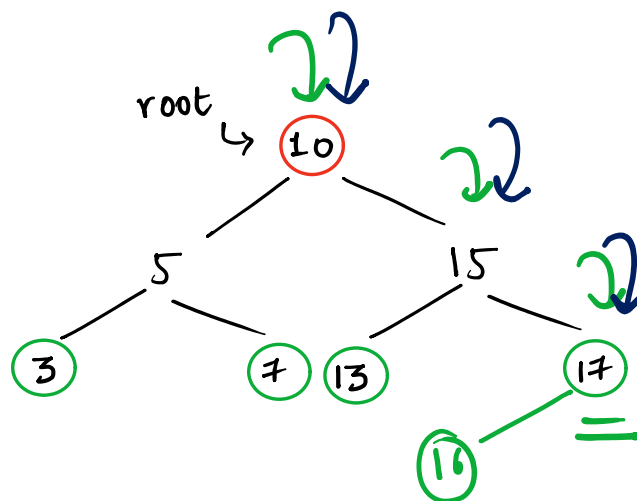
if $\text{key} < \text{root} \rightarrow \text{val}$
 insert the key in the LST } friend
 else // $\text{key} > \text{root} \rightarrow \text{val}$
 insert the key in the RST } friend

↳ BST Search

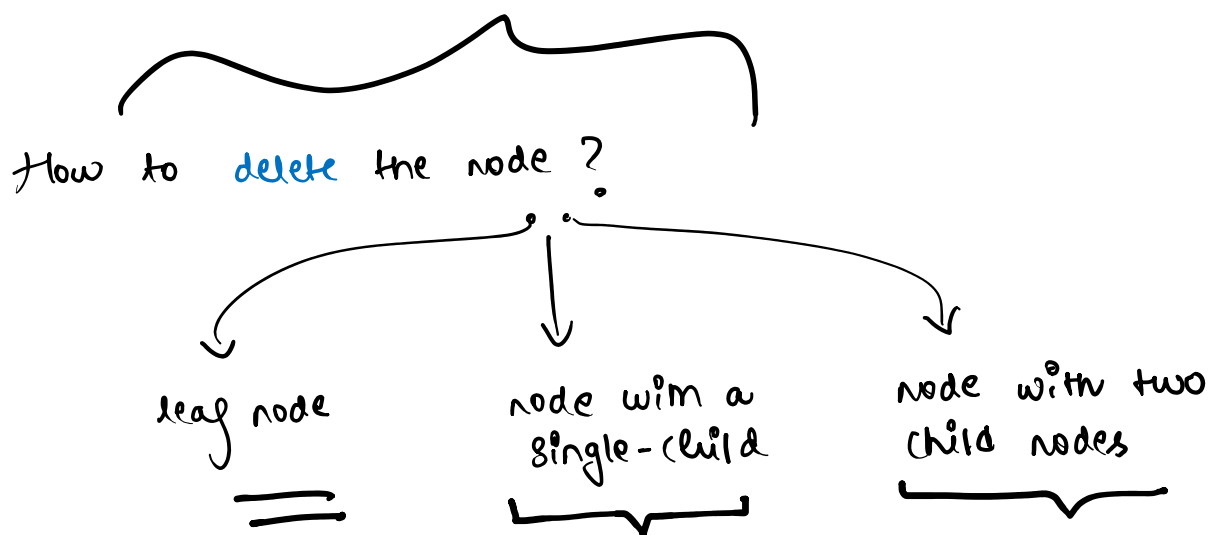
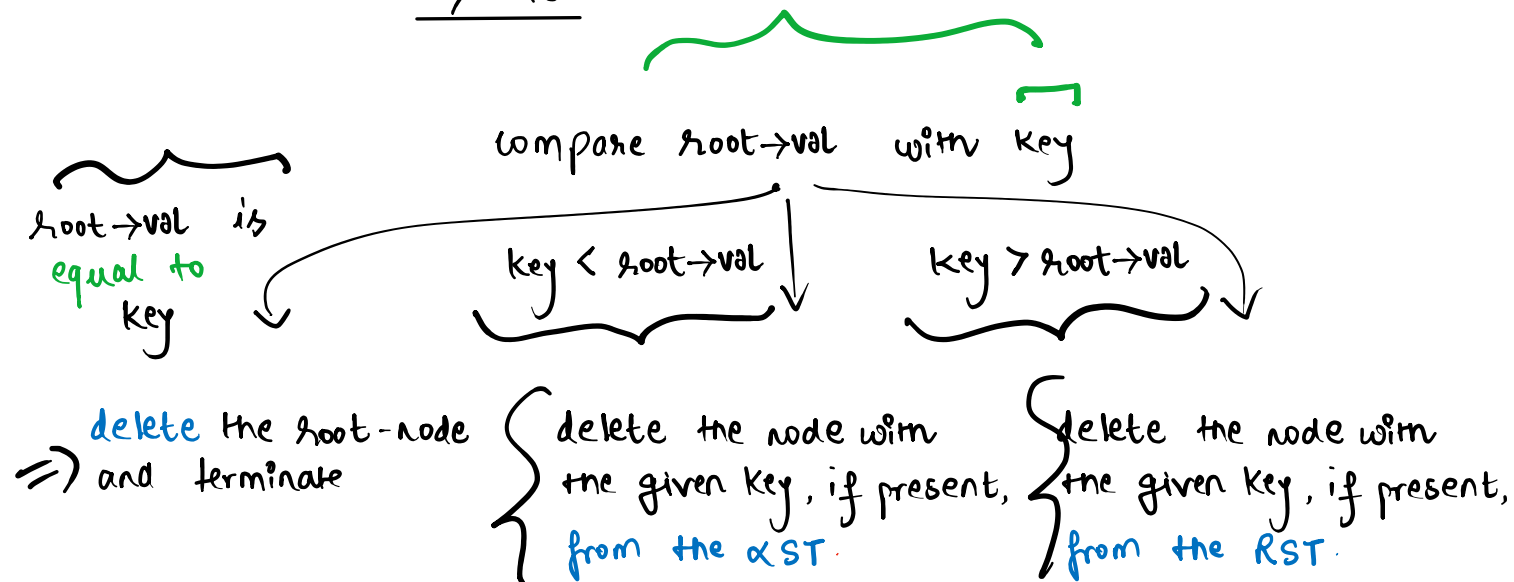
↳ BST: find Minimum



↳ BST: find Maximum

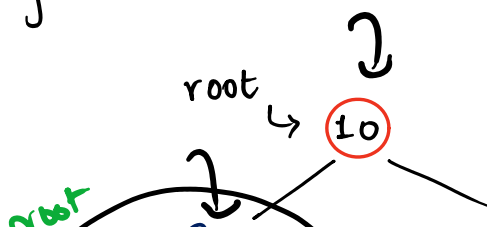


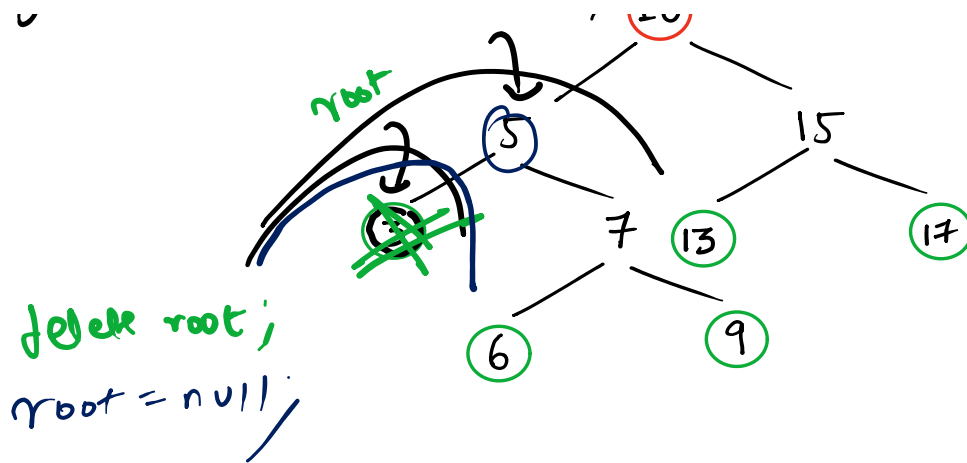
↳ BST Delete



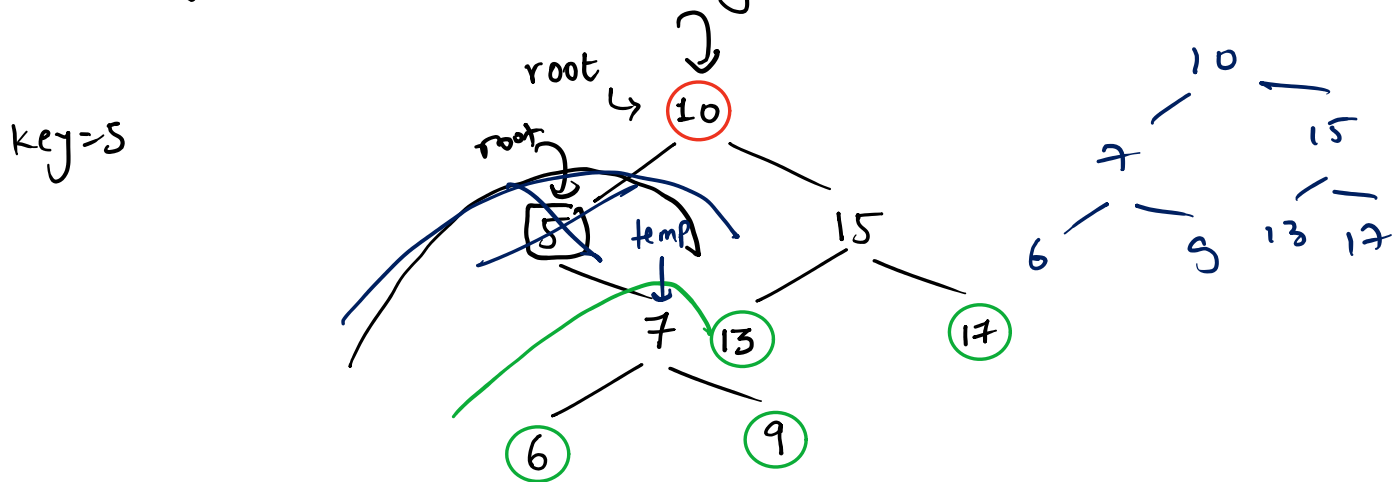
↳ Deleting a leaf node

key = 3





↳ Deleting a node with a single-child



↳ Deleting a node with a two-child nodes



6 7 9 10 13 15 17 =

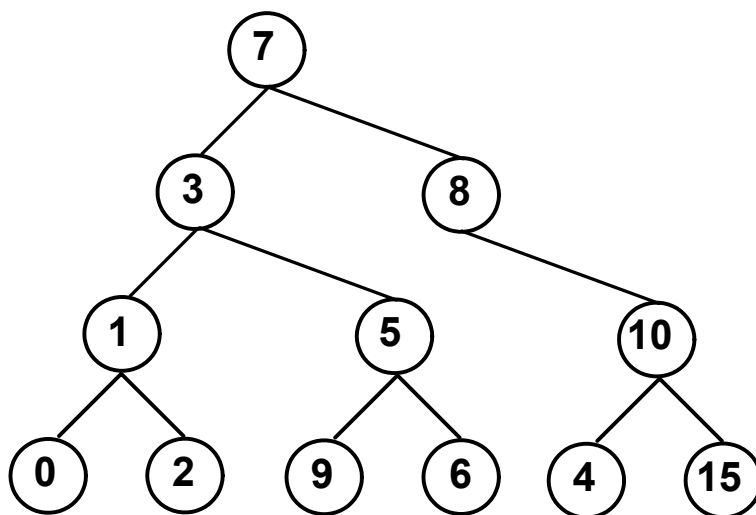
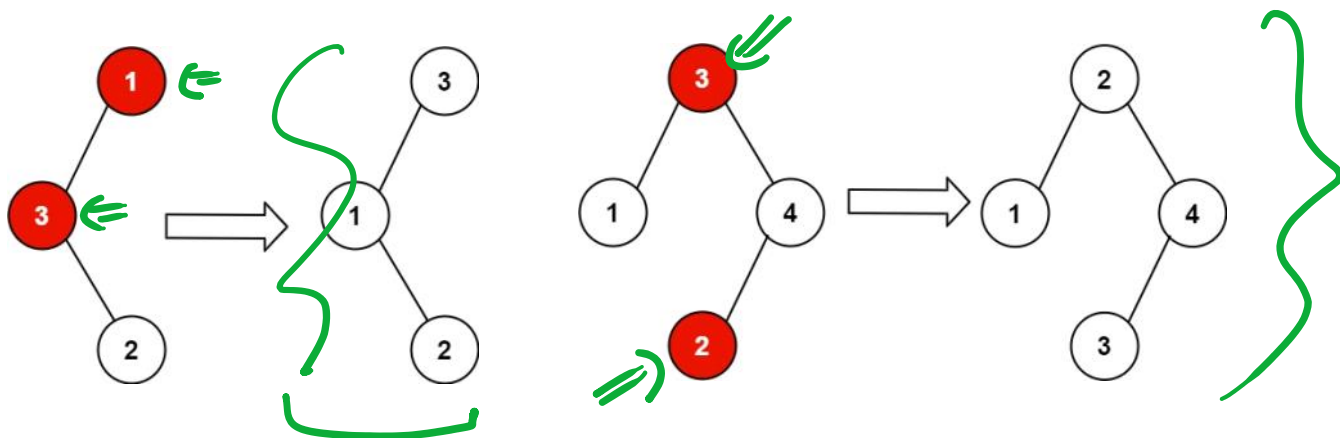
in order
pred

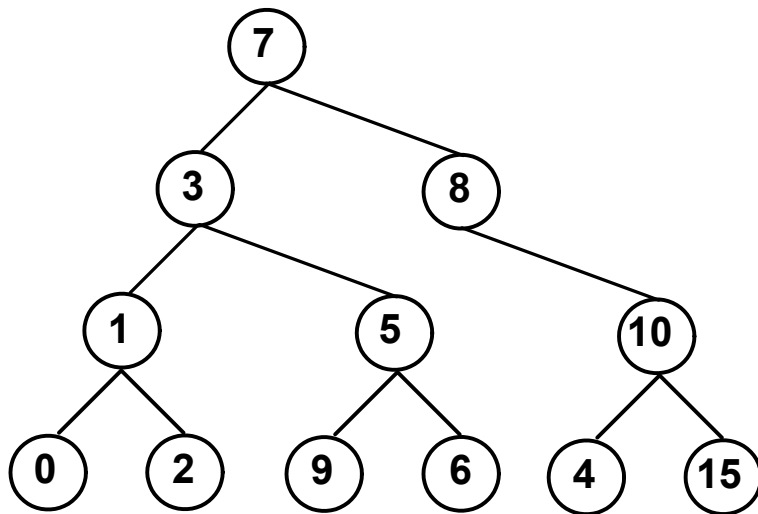
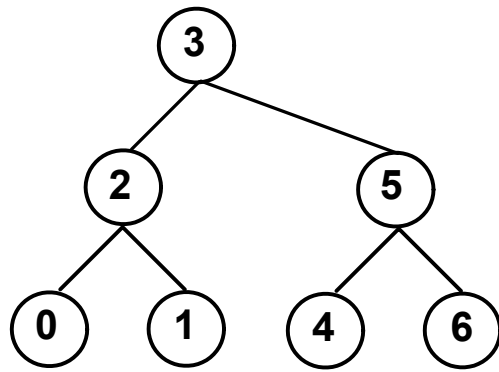
in order
succ

[HW] BST - Recover Binary Search Tree

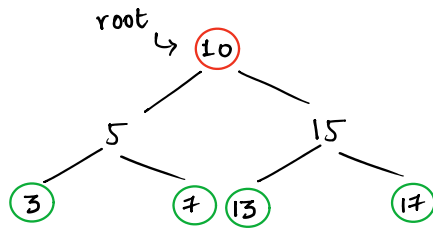
Recover Binary Search Tree

Given the **root** of a binary search tree (BST), where the values of **exactly two** nodes of the tree were **swapped by mistake**. Design an algorithm to **recover** the BST without changing the structure of the tree.

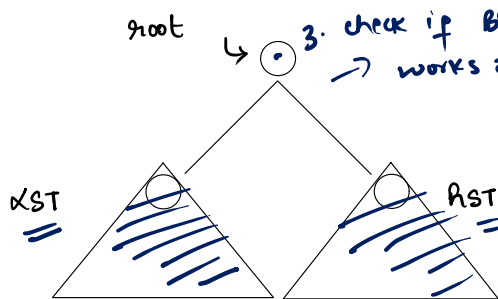




↳ Check if a BT is a BST?

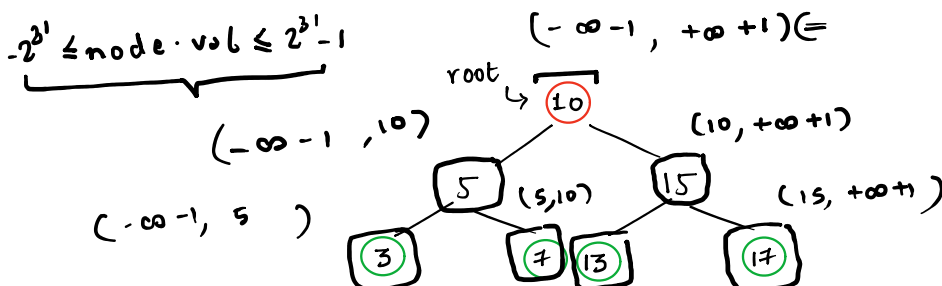
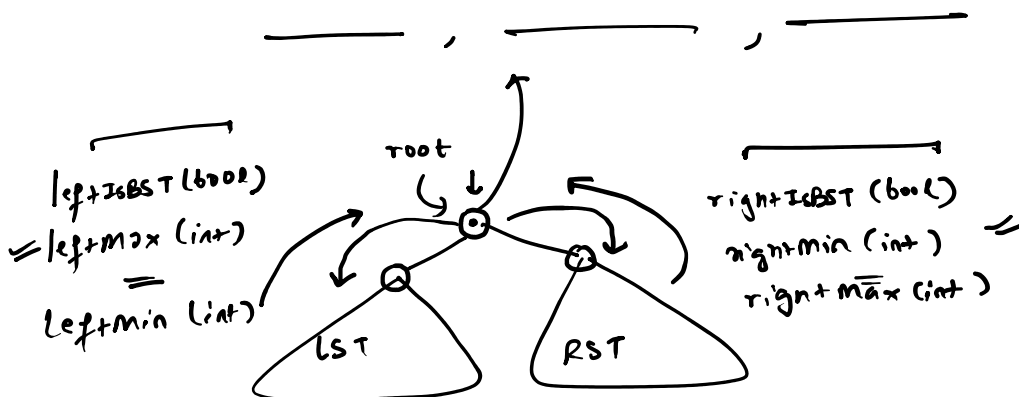


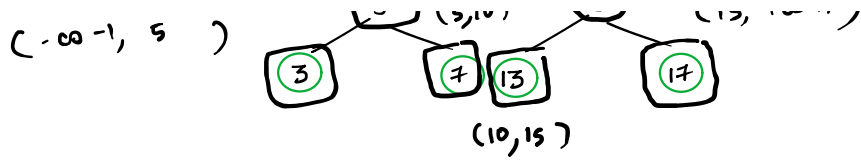
check if given tree is a BST



1. check if LST is a BST
→ friend
leftIsBST (bool)
2. check if RST is a BST
→ friend
rightIsBST (bool)

$\text{root} \rightarrow \text{val} > \text{leftMax}$
 and
 $\text{root} \rightarrow \text{val} < \text{rightMin}$

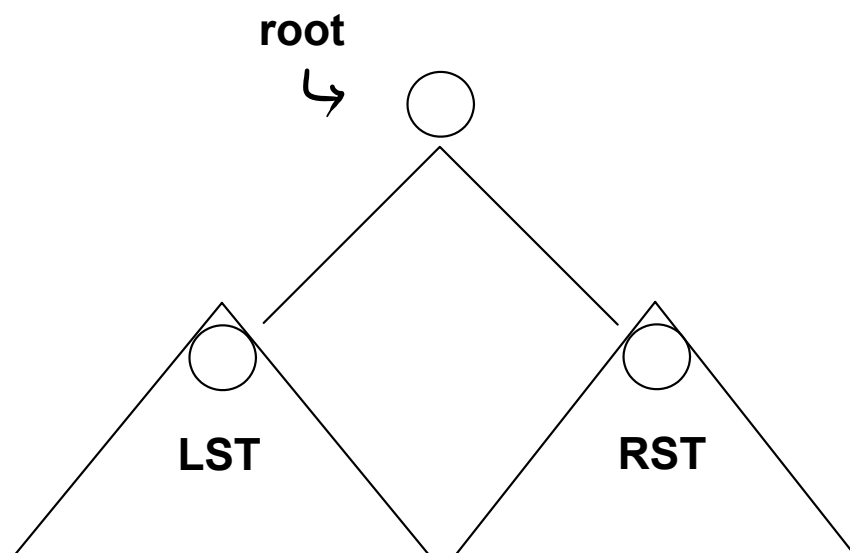
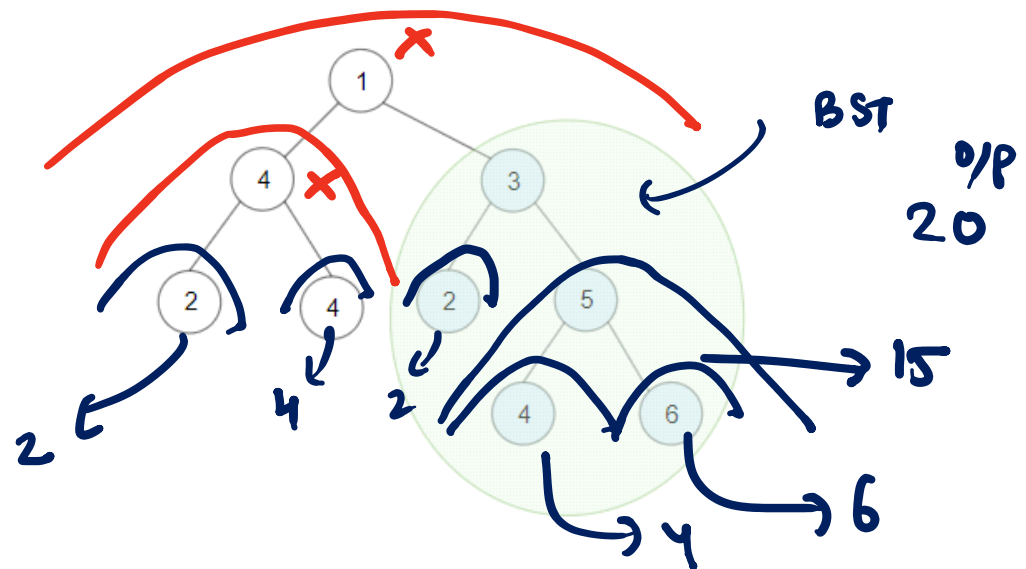




[R][HW] BST - Maximum Sum BST in Binary Tree

Maximum Sum BST in Binary Tree

Given **root** of a binary tree, design an algorithm to find the **maximum** sum of all keys of any sub-tree which is also a Binary Search Tree (BST).



[HW] BST to Sorted Linked List

